

CSCE 411

Dynamic Programming Practice

Describe and analyze efficient bottom-up dynamic programming algorithms for the following problems:

1. Given an array $A[1, \dots, n]$ of integers, compute the length of the longest decreasing subsequence of A .

Solution (Two parameters): Let $LIS(i, j)$ denote the length of the longest decreasing subsequence of $A[i, \dots, n]$ where every element is smaller than $A[j]$. We give the following recurrence:

$$LIS(i, j) = \begin{cases} 0 & \text{if } i > n \\ LIS(i + 1, j) & \text{if } j \leq n \text{ and } A[i] \geq A[j] \\ \max\{LIS(i + 1, j), 1 + LIS(i + 1, i)\} & \text{otherwise} \end{cases}$$

Here our recurrence represents the choice of whether or not to include $A[i]$ in our subsequence. If $A[i] \geq A[j]$, we have no choice, we have to skip $A[i]$. If $A[i] < A[j]$, we can include it, in which case we increase the length of our sequence by 1, but change our upper bound on future elements to $A[i]$, or we can skip it. We choose whichever option gives a longer subsequence. To solve the original problem, we need to compute $\max_{1 \leq i \leq n} (1 + LIS(i + 1, i))$.

We can memoize the function LIS into a 2D array $D[1, \dots, n + 1][1, \dots, n]$. Each entry $D[i][j]$ depends only on entries in the next row $D[i + 1][*]$, so we can fill the array from bottom to top (we arbitrarily choose to fill each row from left to right).

```
LIS( $A[1, \dots, n]$ ):
   $D \leftarrow$  a 2D array of size  $(n + 1) \times n$ 
  For  $j$  from 1 to  $n$            «Base cases»
     $D[n + 1][j] \leftarrow 0$ 

  For  $i$  from  $n$  down to 1
    For  $j$  from 1 to  $n$ 
      If  $A[i] \geq A[j]$ 
         $D[i][j] \leftarrow D[i + 1][j]$ 
      Else
         $D[i][j] \leftarrow \max\{D[i + 1][j], 1 + D[i + 1][i]\}$ 

  Return  $\max_{1 \leq i \leq n} (1 + D[i + 1][i])$ 
```

Our array has $\Theta(n^2)$ entries, and we can compute each one in $\Theta(1)$ time (computing a new entry requires comparing $A[i]$ and $A[j]$ and then either copying a value we already computed or taking the max of two values we already computed and copying one of them, all of which are constant time operations). Therefore the total runtime to fill in our table is $\Theta(n^2) \cdot \Theta(1) = \Theta(n^2)$. Computing the final answer from our table requires taking the max over n entries which takes $\Theta(n)$ time. Thus, our algorithm runs in $\Theta(n^2)$ time. We note that the space complexity is $\Theta(n^2)$.

Solution (One parameter): If we want to be clever, we can reduce the space complexity of our DP algorithm by defining our subproblems differently.

Let $LIS(i)$ denote the length of the longest decreasing subsequence of $A[i, \dots, n]$ that begins with $A[i]$. We give the following recurrence:

$$LIS(i) = 1 + \max_{i < j \leq n \text{ with } A[j] < A[i]} LIS(j).$$

Here our recurrence represents choosing the next element in the sequence after $A[i]$. The valid choices are all elements after index i which are less than $A[i]$, and we pick whichever one gives us the longest subsequence. We define $\max \emptyset = 0$ so that this recurrence covers the base cases. To solve the original problem, we need to compute $\max_{1 \leq i \leq n} LIS(i)$.

We can memoize the function LIS into a 1D array $D[1, \dots, n]$. Each entry $D[i]$ depends only subsequent entries, so we can fill the array from back to front.

```

LIS( $A[1, \dots, n]$ ):
   $D \leftarrow$  a 1D array of size  $n$ 
  For  $i$  from  $n$  down to 1
     $max\_len \leftarrow 0$ 
    For  $j$  from  $i + 1$  to  $n$ 
      If  $A[j] < A[i]$  and  $D[j] > max\_len$ 
         $max\_len \leftarrow D[j]$ 
     $D[i] \leftarrow 1 + max\_len$ 
  Return  $\max_{1 \leq i \leq n} D[i]$ 

```

Our array has $\Theta(n)$ entries, but each one takes $\Theta(n)$ time to compute, since computing $D[i]$ requires looping over all of $A[i + 1, \dots, n]$. Therefore the total runtime to fill in our table is $\Theta(n) \cdot \Theta(n) = \Theta(n^2)$. Computing the final answer from our table requires taking the max over n entries which takes $\Theta(n)$ time. Thus, our algorithm runs in $\Theta(n^2)$ time. We note that the space complexity is $\Theta(n)$.

2. Given an array $A[1, \dots, n]$ of integers, compute the length of the longest alternating subsequence of A . We say that a sequence $x_1, \dots, x_k$ is *alternating* if $x_i < x_{i+1}$ for every odd index $1 \leq i \leq k - 1$ and $x_i > x_{i+1}$ for every even index $1 \leq i \leq k - 1$.

Solution: Let $LAS(i, inc)$ denote the length of the longest subsequence of $A[i, \dots, n]$ beginning with $A[i]$ which alternates between increasing and decreasing and if inc is True, then the subsequence must begin by increasing, while if inc is False, the subsequence must begin by decreasing. We give the following recurrence:

$$LAS(i, inc) = \begin{cases} 1 + \max_{i < j \leq n \text{ with } A[j] > A[i]} LAS(j, \text{False}) & \text{if } inc = \text{True} \\ 1 + \max_{i < j \leq n \text{ with } A[j] < A[i]} LAS(j, \text{True}) & \text{if } inc = \text{False} \end{cases}$$

Here our recurrence represents choosing the next element in the sequence after $A[i]$. If our subsequence must start by increasing, then the valid choices are subsequent elements which are larger than $A[i]$, and the rest of the sequence starts out decreasing. If our subsequence must start by decreasing, then the valid choices are subsequent elements which are smaller than $A[i]$, and the rest of the sequence starts out increasing. Once again we define $\max \emptyset = 0$ so that this recurrence covers the base cases. To solve the original problem, we need to compute $\max_{1 \leq i \leq n} LAS(i, \text{True})$.

We can memoize the function LAS into a 2D array $D[1, \dots, n][1, 2]$. Here $D[i][1]$ will store $LAS(i, \text{False})$ and $D[i][2]$ will store $LAS(i, \text{True})$. Each entry $D[i][inc]$ depends only on entries in later rows, so we can fill the array from bottom to top.

```

LAS(A[1, ..., n]):
  D ← a 2D array of size n × 2
  For i from n down to 1
    max_inc ← 0
    max_dec ← 0
    For j from i + 1 to n
      If A[j] < A[i] and D[j][2] > max_inc
        max_inc ← D[j][2]
      If A[j] > A[i] and D[j][1] > max_dec
        max_dec ← D[j][1]
    D[i, 1] ← 1 + max_inc
    D[i, 2] ← 1 + max_dec
  Return max_{1 ≤ i ≤ n} D[i, 2]

```

Our array has $\Theta(n)$ entries, but each one takes $\Theta(n)$ time to compute, since computing $D[i, inc]$ requires looping over all of the later rows of A . Therefore the total runtime to fill in our table is $\Theta(n) \cdot \Theta(n) = \Theta(n^2)$. Computing the final answer from our table requires taking the max over n entries which takes $\Theta(n)$ time. Thus, our algorithm runs in $\Theta(n^2)$ time. We note that the space complexity is $\Theta(n)$.

3. Given an array $A[1, \dots, n]$ of integers, compute the length of the longest palindrome subsequence of A . We say that a sequence $x_1, \dots, x_k$ is a *palindrome* if $x_i = x_{k-i+1}$ for every index $1 \leq i \leq k$

Solution: Let $LPS(i, j)$ denote the length of the longest palindrome subsequence of $A[i, \dots, j]$. We give the following recurrence:

$$LIS(i, j) = \begin{cases} 0 & \text{if } i > j \\ 1 & \text{if } i = j \\ \max\{LPS(i+1, j), LPS(i, j-1)\} & \text{if } i < j \text{ and } A[i] \neq A[j] \\ \max\{2 + LPS(i+1, j-1), \\ LPS(i+1, j), LPS(i, j-1)\} & \text{otherwise} \end{cases}$$

Here our recurrence represents the choice of whether to include the first and/or last elements of our current range in the subsequence. If $A[i] \neq A[j]$, a palindrome subsequence cannot possibly include both of them, so we try throwing out each and see which gives a longer palindrome subsequence. If $A[i] = A[j]$ we also have the option of including $A[i]$ and $A[j]$ as the first and last elements of our subsequence in which case the rest of the subsequence just needs to be a palindrome subsequence of $A[i+1, \dots, j-1]$ (it's possible to prove that including these two elements in the subsequence when possible is always optimal, but simplifying the recurrence in this way does not improve the asymptotic runtime of our algorithm). To solve the original problem, we need to compute $LPS(1, n)$.

We can memoize the function LPS into a 2D array $D[1, \dots, n][1, \dots, n]$. Each entry $D[i][j]$ depends only on entries $D[i+1][j]$, $D[i][j-1]$, and $D[i+1][j-1]$. Therefore we fill in the rows from bottom to top and within each row we fill in the entries from left to right.

$LPS(A[1, \dots, n])$:

$D \leftarrow$ a 2D array of size $n \times n$

For i from n down to 1

 For j from 1 to n

 If $i > j$:

$D[i][j] \leftarrow 0$

 Else If $i = j$:

$D[i][j] \leftarrow 1$

 Else If $A[i] \neq A[j]$:

$D[i][j] \leftarrow \max\{D[i+1][j], D[i][j-1]\}$

 Else:

$D[i][j] \leftarrow \max\{2 + D[i+1, j-1], D[i+1][j], D[i][j-1]\}$

Return $D[1, n]$

Our array has $\Theta(n^2)$ entries, and we can compute each one in $\Theta(1)$ time (computing a new entry requires comparing a constant number of existing entries). Therefore the total runtime to fill in our table is $\Theta(n^2) \cdot \Theta(1) = \Theta(n^2)$. Computing the final answer from our table requires looking at a single entry which takes $\Theta(1)$ time. Thus, our algorithm runs in $\Theta(n^2)$ time. We note that the space complexity is $\Theta(n^2)$.

4. You are at a fancy sushi restaurant which uses a massive conveyor belt to display the sushi. You have come prepared to maximize your enjoyment of this dining experience, so you assembled a list of the n different pieces of sushi $s_1, \dots, s_n$ that will be served

on the conveyor belt, in the order that they will arrive in front of you. For each piece of sushi, you have recorded:

- The enjoyment e_i that you will receive from eating s_i (You have found a way to quantify this, don't ask me how)
- The fullness f_i that you will receive from eating s_i , measured in how many piece of sushi you will need to let pass by before you can eat another (assume that this includes the time it takes you to eat s_i).

Each piece of sushi will only pass by you once, and you must immediately choose whether to eat it or let it pass.

Give an algorithm that takes this information about the sushi restaurant and determines the maximum total enjoyment that you can receive from this dining experience.

Solution: Let $ME(i)$ denote the maximum amount of enjoyment that you can obtain from $s_i, \dots, s_n$, i.e. the maximum enjoyment you can obtain if you let the first $i - 1$ pieces of sushi pass you by. We give the following recurrence:

$$ME(i) = \begin{cases} 0 & \text{if } i > n \\ \max(ME(i + 1), e_i + ME(i + f_i)) & \text{otherwise} \end{cases}$$

Here our recurrence represents the choice of whether or not to eat s_i . If we skip s_i , the enjoyment we obtain is the same as if we skipped the first i pieces of sushi. If we eat s_i , we gain e_i units of enjoyment, but we will be full until s_{i+f_i} , so the maximum enjoyment we can obtain from the rest of the dinner is $ME(i + f_i)$. The answer to the original problem is $ME(1)$.

We can memoize the function ME into a 1D array $D[1, \dots, n + 1]$. Each entry $D[i]$ depends only on later entries, so we can fill the array from back to front.

```

ME( $e_1, \dots, e_n, f_1, \dots, f_n$ ):
   $D \leftarrow$  an array of size  $n + 1$ 
   $D[n + 1] \leftarrow 0$ 
  For  $i$  from  $n$  down to 1
     $D[i] \leftarrow \max(D[i + 1], e_i + D[\min(i + f_i, n + 1)])$ 
  Return  $D[1]$ 

```

Our array has $\Theta(n)$ entries, and we can compute each one in $\Theta(1)$ time. Thus, our algorithm runs in $\Theta(n)$ time. We note that the space complexity is $\Theta(n)$.